BLE Offline Messaging MVP — Test Scenarios

> ***What you need:*** *Two iOS devices (or one device + Simulator on same Mac — though MPC over Simulator can be flaky). Both must be on the* ***same Wi-Fi*** *or have* ***Bluetooth enabled****. The app uses Apple's MultipeerConnectivity (MPC), which works over Wi-Fi and Bluetooth automatically.*

Architecture Quick Recap

File	Role
Message.swift	Data model (`Message`) + wire-format (`MessagePayload`)
MessageStore.swift	JSON file persistence (Documents/messages.json)
ConnectivityService.swift	MPC advertiser + browser + session management
ChatViewModel.swift	Compose → Queue → Send → Receive coordinator
ContentView.swift	SwiftUI chat UI (bubbles, compose bar, connection badge)

Category 1 — Connection Lifecycle

Scenario 1.1: Auto-Discovery & Auto-Connect

Goal: Verify two devices discover each other and connect automatically with zero user interaction.

Prerequisites:

- Fresh install on **Device A** and **Device B**
- Both on same Wi-Fi network and/or Bluetooth enabled

Steps:

#	Action	What to observe
1	Launch the app on **Device A** only	Connection badge (top-right) shows 🟠 **"Searching..."**
2	Launch the app on **Device B**	Device B also shows 🟠 **"Searching..."** briefly
3	Wait 5-15 seconds	Both devices' badges change to 🟢 **"1 peer"**

Expected Result:

- Both badges turn **green** with text **"1 peer"**
- No pairing dialogs, no user taps required

Code Explanation:

When the app launches, BLEOfflineMVPApp.swift creates a `@StateObject` `ChatViewModel`. Inside ChatViewModel.init(), the last line calls:

```
connectivity.start()   // line 62
```

This calls ConnectivityService.start(), which does two things simultaneously:

```
advertiser.startAdvertisingPeer()   // "I'm here, find me"
browser.startBrowsingForPeers()     // "Let me find others"
```

and sets `connectionState = .searching` → the badge shows orange.

When Device B's browser discovers Device A, the browser(_:foundPeer:withDiscoveryInfo:) delegate fires and **auto-invites** the peer:

```
browser.invitePeer(peerID, to: session, withContext: nil, timeout: 30)
```

On the receiving side, advertiser(_:didReceiveInvitationFromPeer:...) **auto-accepts** every invitation:

```
invitationHandler(true, session)   // zero user friction
```

Once the MPC session is established, session(_:peer:didChange:) fires with `.connected`, appends the peer to `connectedPeers`, and updates `connectionState` to `.connected(peerCount: 1)`. The badge reflects this via the Combine pipeline in ChatViewModel.setupObservers():

```
connectivity.$connectionState
    .receive(on: DispatchQueue.main)
    .assign(to: &$connectionState)
```

Scenario 1.2: Disconnection & Reconnection

Goal: Verify the badge updates on disconnect and peers auto-reconnect.

Prerequisites:

- Both devices connected (🟢 badge from Scenario 1.1)

Steps:

#	Action	What to observe
1	On **Device B**, toggle **Airplane Mode ON** (or kill the app)	Device A badge changes from 🟢 "1 peer" → 🟠 "Searching..."
2	Wait 5 seconds	Device A stays on 🟠 "Searching..."
3	On **Device B**, toggle **Airplane Mode OFF** (or relaunch the app)	After ~10-15 sec, Device A badge returns to 🟢 "1 peer"

Expected Result:

- State transitions: `connected` → `searching` → `connected`
- No manual intervention required to reconnect

Code Explanation:

When Device B goes offline, the MPC session fires handleStateChange() with `.notConnected`:

```
case .notConnected:
    connectedPeers.removeAll { $0 == peer }
```

Since `connectedPeers` is now empty and `isAdvertising` / `isBrowsing` are still `true`, the state falls into:

```
if connectedPeers.isEmpty {
    connectionState = (isAdvertising || isBrowsing) ? .searching : .idle
}
```

This sets the badge back to orange "Searching...". The advertiser/browser are **never stopped**, so when Device B comes back, MPC re-discovers and re-invites automatically through the same `foundPeer → invitePeer → auto-accept` flow.

Category 2 — Online Messaging (Both Devices Connected)

Scenario 2.1: Send a Message While Connected

Goal: Verify a message sent while connected is delivered instantly and shows a ✅ checkmark.

Prerequisites:

- Both devices connected (🟢🟢)

Steps:

#	Action	What to observe
1	On **Device A**, type `Hello from A!` in the text field	Send button (blue ↑ circle) becomes enabled
2	Tap the **Send** button	**Device A**: Blue bubble appears, right-aligned, with 🟢 **checkmark** (✓) and timestamp. Text field clears.
3	Look at **Device B**	**Device B**: Gray bubble appears, left-aligned, with Device A's name above it and a timestamp below.

Expected Result:

- **Device A** sees: `Hello from A!` → blue bubble → ✓ green checkmark (status = `.sent`)
- **Device B** sees: `Hello from A!` → gray bubble → sender name shown above bubble
- Message appears at the **bottom** of the chat on both devices

Code Explanation:

When you tap Send, ChatViewModel.sendMessage() runs:

```
let message = Message(
    text: text,
    senderName: connectivity.displayName,
    isMine: true,
    status: .queued       // ← starts as queued
)

composeText = ""           // ← clears text field
insertMessage(message)     // ← adds to in-memory array + persists
```

Since `connectedPeerCount > 0`, it immediately calls sendOverWire():

```
let payload = MessagePayload(from: message)  // strips isMine, status
let data = try encoder.encode(payload)
try connectivity.sendToAll(data)             // sends to all peers
updateMessageStatus(id: message.id, to: .sent)  // ← queued → sent ✓
```

The `MessagePayload` is a slimmed-down wire format (Message.swift L51-62) that excludes `isMine` and `status` — those are local-only fields.

On Device B, the MPC session triggers session(:didReceive:fromPeer:), which publishes the raw data through the `dataReceived` Combine subject. The ChatViewModel's observer in setupObservers() picks it up:

```
connectivity.dataReceived
    .receive(on: DispatchQueue.main)
    .sink { [weak self] data, _ in
        self?.handleReceivedData(data)
    }
```

handleReceivedData() decodes the payload and creates a `Message` with `isMine: false, status: .received`:

```
let payload = try decoder.decode(MessagePayload.self, from: data)
let message = payload.toMessage()  // isMine=false, status=.received
insertMessage(message)
```

In ContentView.swift, the `MessageBubble` renders:

- **Own messages** (`isMine == true`): Blue background, right-aligned, with status icon
- **Received messages** (`isMine == false`): Gray background, left-aligned, sender name shown

The statusIcon `@ViewBuilder` maps:

- `.queued` → 🟠 clock icon
- `.sent` → 🟢 checkmark
- `.delivered` → 🟢 filled checkmark circle
- `.received` → nothing (it's someone else's message)

Scenario 2.2: Bidirectional Conversation

Goal: Verify messages from both sides interleave correctly in arrival order.

Prerequisites:

- Both devices connected (🟢)

Steps:

#	Action	Device A screen	Device B screen
1	**Device A** sends `Message 1`	Blue: "Message 1" ✓	Gray: "Message 1"
2	**Device B** sends `Reply 1`	Gray: "Reply 1"	Blue: "Reply 1" ✓
3	**Device A** sends `Message 2`	Blue: "Message 2" ✓	Gray: "Message 2"
4	**Device B** sends `Reply 2`	Gray: "Reply 2"	Blue: "Reply 2" ✓

Expected Result:

- Both devices show all 4 messages in the **order they were received/composed**
- Own messages are blue (right), remote messages are gray (left)
- Chat auto-scrolls to the bottom on each new message

Code Explanation:

Messages are appended in arrival order via insertMessage():

```
private func insertMessage(_ message: Message) -> Bool {
    guard !knownMessageIDs.contains(message.id) else { return false }  // dedup
    knownMessageIDs.insert(message.id)
    messages.append(message)    // ← always appends at the end
    persistToDisk()
```

```
    return true
}
```

The `messages` array is simply appended to — there's **no re-sorting by timestamp**. This was a deliberate design decision (from a prior conversation) to avoid clock-skew issues between devices. Messages appear in the order they arrive at each device.

Auto-scroll is handled in ContentView:

```
.onChange(of: viewModel.messages.count) { _ in
    scrollToBottom(proxy: proxy)
}
```

Scenario 2.3: Duplicate Message Prevention

Goal: Verify the same message is never displayed twice.

Prerequisites:

- Both devices connected (🟢)

Steps:

#	Action	What to observe
1	**Device A** sends `Unique message`	Appears once on both devices
2	Disconnect Device B (Airplane Mode)	—
3	Reconnect Device B	The message is **NOT** duplicated on either device

Expected Result:

- Each message appears exactly once on each device, even across reconnections

Code Explanation:

Deduplication works at two levels:

1. **In-memory** (ChatViewModel.insertMessage()): Uses a `Set<UUID>` called `knownMessageIDs` for O(1) lookup:

   ```
   guard !knownMessageIDs.contains(message.id) else { return false }
   ```

2. **On-disk** (MessageStore.appendMessage()): Checks the persisted array before appending:

   ```
   guard !messages.contains(where: { $0.id == message.id }) else {
       return messages
   }
   ```

Each `Message` is created with a `UUID` (Message.swift L15): `let id: UUID`, and this UUID travels through the wire via `MessagePayload`. So even if a message is re-sent during outbox flush, the receiver already has the ID and silently drops it.

Category 3 — Offline-First (Queuing & Flush)

Scenario 3.1: Compose While Offline → Auto-Deliver on Reconnect

Goal: Verify messages composed without any peer connection are queued and automatically sent when a peer connects.

Prerequisites:

- Launch app on **Device A only** (no peer in range)

Steps:

#	Action	What to observe on Device A
1	Verify badge shows 🟠 "Searching..."	No peers connected
2	Type `Offline msg 1` and tap Send	Blue bubble appears with 🟠 **clock icon** (status = `.queued`)
3	Type `Offline msg 2` and tap Send	Second blue bubble with 🟠 clock icon
4	Now launch app on **Device B**	Wait for connection (🟢)
5	Watch **Device A**	Both clock icons change to 🟢 **checkmarks** (status = `.sent`)
6	Watch **Device B**	Both messages appear in gray bubbles

Expected Result:

- Messages composed offline show a **clock icon** (queued)
- Once connected, status changes to **checkmark** (sent) automatically
- Receiver gets all queued messages in order

Code Explanation:

When you send a message while offline, sendMessage() creates it with `status: .queued` and calls `insertMessage()` to persist. However, the `if connectedPeerCount > 0` check on line 146 **fails**, so `sendOverWire()` is never called — the message just sits in the array with a clock icon.

The magic happens in setupObservers(). There's a Combine observer on `connectedPeers`:

```
connectivity.$connectedPeers
    .receive(on: DispatchQueue.main)
    .sink { [weak self] peers in
        if !peers.isEmpty {
            self?.flushOutbox()   // ← triggers on first peer connect
        }
    }
```

flushOutbox() finds all queued messages:

```
let queued = messages.filter { $0.isMine && $0.status == .queued }
for message in queued {
    sendOverWire(message)   // encode, send, mark as .sent
}
```

Each queued message is then encoded into a `MessagePayload`, sent via MPC, and marked `.sent` — causing the clock icon to flip to a checkmark in the UI.

Scenario 3.2: Both Devices Compose Offline → Mutual Delivery

Goal: Verify that when both devices queue messages offline, they exchange them upon connecting.

Prerequisites:

- Both devices launched separately with **no connection** to each other (e.g., Airplane Mode on both)

Steps:

#	Action	What to observe
1	**Device A** (offline): Send `A offline 1`, `A offline 2`	Blue bubbles with 🟠 clock icons
2	**Device B** (offline): Send `B offline 1`, `B offline 2`	Blue bubbles with 🟠 clock icons
3	Disable Airplane Mode on both (or bring into range)	Both devices connect (🟢)
4	Wait a few seconds	**Device A**: Clock icons → checkmarks. `B offline 1`, `B offline 2` appear as gray bubbles.
5	Check **Device B**	Clock icons → checkmarks. `A offline 1`, `A offline 2` appear as gray bubbles.

Expected Result:

- Each device's queued messages flush automatically
- Both devices end up with all 4 messages
- Own messages show ✓, received messages show sender name

Code Explanation:

Both devices independently have the same `$connectedPeers` observer. When the MPC connection is established, **both** devices fire `flushOutbox()` near-simultaneously. The `sendOverWire()` on each device sends its queued messages to the other, and the receiver's `handleReceivedData()` appends them. Dedup via `knownMessageIDs` prevents any issues if a message somehow arrives twice.

Category 4 — Persistence & App Lifecycle

Scenario 4.1: Messages Survive App Kill & Relaunch

Goal: Verify all messages (sent, received, queued) are persisted and restored on app relaunch.

Prerequisites:

- At least 3 messages in the chat (some sent, some received)

Steps:

#	Action	What to observe
1	Note the current messages in the chat	Count messages, note their order and statuses
2	**Force-kill** the app (swipe up from app switcher)	—
3	Relaunch the app	All messages reappear in the **same order** with the **same statuses**

Expected Result:

- Message count, order, text, sender names, timestamps, and status icons all match pre-kill state
- Queued messages still show clock icons (they haven't been sent yet)

Code Explanation:

Every mutation to the messages array calls persistToDisk():

```
private func persistToDisk() {
    store.saveMessages(messages)
}
```

Which calls MessageStore.saveMessages():

```
func saveMessages(_ messages: [Message]) {
    let data = try encoder.encode(messages)
    try data.write(to: fileURL, options: .atomic)  // atomic = safe write
}
```

Messages are stored as a JSON array in `Documents/messages.json` using ISO 8601 date encoding.

On relaunch, ChatViewModel.init() loads them back:

```
let loaded = store.loadMessages()
self.messages = loaded
self.knownMessageIDs = Set(loaded.map(\.id))  // rebuild dedup set
```

MessageStore.loadMessages() reads and decodes the JSON file. The order is preserved because JSON arrays maintain insertion order.

Scenario 4.2: Queued Messages Are Flushed After App Relaunch

Goal: Verify that messages queued before an app kill are still sent after relaunch + reconnect.

Prerequisites:

- Device A only (no peer), compose messages offline

Steps:

#	Action	What to observe
1	**Device A** (offline): Send `Survive this!`	Blue bubble with 🟠 clock
2	**Force-kill** the app on Device A	—
3	Relaunch the app on Device A	Message `Survive this!` reappears with 🟠 clock (still queued)
4	Launch app on **Device B**	Devices connect (🟢)
5	Wait a few seconds	**Device A**: Clock → ✓ checkmark. **Device B**: Message appears as gray bubble.

Expected Result:

- Queued status survives app kill
- Outbox flush still works after relaunch because the Combine observer fires on first peer connection

Code Explanation:

The key insight: `status` is a persisted field in the `Message` struct (it's `Codable`). So `.queued` status survives the JSON roundtrip. On relaunch:

1. `store.loadMessages()` returns messages with their persisted `.queued` status
2. `connectivity.start()` begins advertising/browsing

3. When a peer connects, the `$connectedPeers` observer fires `flushOutbox()`
4. `flushOutbox()` filters `messages.filter { $0.isMine && $0.status == .queued }` — finds the surviving queued message
5. Sends it and marks it `.sent`

UI Element Reference

UI Element	Where	Behavior
Connection badge	Navigation bar, trailing	Capsule with colored dot + text. Color/text driven by `ConnectionState` enum
Empty state	Center of chat area	Icon + "No Messages" + instructions. Shown when `messages.isEmpty`
Send button	Bottom-right of compose bar	Blue circle with ↑ arrow. Disabled when text field is empty/whitespace-only
Blue bubble	Right-aligned	Own messages (`isMine == true`)
Gray bubble	Left-aligned	Received messages (`isMine == false`), with sender name above
Clock icon 🟠	Below own message	Status = `.queued` (not yet sent)
Checkmark 🟢	Below own message	Status = `.sent` (delivered to peer)
Filled checkmark 🟢	Below own message	Status = `.delivered` (future use — not currently triggered)

Quick Checklist for Verification

- [ ] **1.1** Two devices auto-discover and show green badge
- [ ] **1.2** Disconnect → orange badge → reconnect → green badge
- [ ] **2.1** Send while connected → instant delivery → checkmark on sender, bubble on receiver
- [ ] **2.2** Bidirectional messages interleave correctly on both screens
- [ ] **2.3** Same message never appears twice (UUID dedup)
- [ ] **3.1** Offline compose → clock icon → connect → auto-flush → checkmark
- [ ] **3.2** Both offline → connect → mutual exchange of all queued messages
- [ ] **4.1** App kill + relaunch → all messages restored with correct statuses
- [ ] **4.2** Queued messages survive app kill and flush on next reconnect